

HOSPITAL PATIENT MANAGEMENT SYSTEM

1. Abstract

The Hospital Patient Management System is a desktop-based application developed to store, manage, and retrieve basic patient information efficiently. The system demonstrates the practical use of Python programming for developing a graphical desktop application integrated with a relational database. It uses Tkinter to provide a simple and user-friendly graphical interface and SQLite3 for storing patient records locally. The project demonstrates important concepts such as database connectivity, CRUD operations, event-driven programming, and record management. Through this system, users can register patients, search for existing records, view patient information, update or delete records, and maintain basic diagnosis and doctor-related details. The application is designed mainly for academic and small-scale departmental use and provides a foundation for developing a more advanced hospital management system in the future.

2. Introduction

Managing patient records manually can be a time-consuming and difficult task. Traditional methods generally depend on physical registers, files, or separate documents for maintaining patient information. Such methods can result in duplicate entries, calculation or recording mistakes, difficulty in searching for information, and the possibility of losing or damaging records. To overcome these problems, computerized record management provides a more organized and efficient approach.

The Hospital Patient Management System provides a simple computerized solution for maintaining patient records. The application allows users to enter, view, search, update, and delete information through a graphical user interface without requiring them to directly write SQL commands. Patient information is stored in a structured SQLite database, which allows the records to remain available even after the application is closed. The project therefore provides a practical demonstration of how Python, Tkinter, and SQLite can be combined to develop a useful desktop management application.

3. Problem Statement

The traditional manual method of maintaining hospital patient records requires physical registers or scattered files. Searching for a particular patient's information can take considerable time, especially when the number of records increases. Manual record management may also result in duplicate records, incorrect entries, difficulty in updating information, and inefficient storage.

The proposed Hospital Patient Management System addresses these problems by storing patient information in a structured SQLite database. A user-friendly graphical interface allows the user to perform common record-management operations quickly

and easily. The system is intended to reduce dependency on paper-based records and provide convenient access to stored patient information.

4. Objectives

The main objective of the project is to develop a simple and efficient desktop application for managing patient information. The system is designed to digitally store patient registration details and provide facilities for searching patients using information such as name, phone number, or patient ID. It also maintains basic diagnosis and doctor-related information. By computerizing the record-management process, the system reduces repetitive manual work and provides faster access to stored information. Another objective is to demonstrate the practical implementation of Python programming, graphical user interface development, and database management in a real-world application.

5. Scope of the Project

The scope of the Hospital Patient Management System is limited to basic patient record management suitable for a small-scale academic, laboratory, or departmental environment. The application focuses on local data storage and provides simple operations such as patient registration, searching, displaying, and deleting records. It is designed to be easy to use and can operate on a standard computer with Python installed. Although the current system provides only basic functionality, its structure can be extended in the future by adding features such as appointment scheduling, billing, prescription management, doctor management, authentication, and reporting.

6. Technologies and System Requirements

The application is developed using Python 3 as the main programming language. Tkinter is used to create the graphical user interface, while SQLite3 is used as the database management system. The project can be developed and executed using common Python development environments such as VS Code, IDLE, or PyCharm.

The hardware requirements are minimal. A basic computer or laptop with approximately 2 GB of RAM and a small amount of free disk space is sufficient for running the application. The software requirements include Python 3.x and the standard Python libraries required by the project. Since SQLite is a lightweight database system and Tkinter is commonly included with Python installations, no separate database server is required.

7. Functional Modules

The system consists of several basic functional modules. The **Patient Registration Module** allows users to enter and save patient information. The **Patient Search Module** enables users to locate records using details such as patient name, phone number, or patient ID. The **Patient Record Display Module** displays stored records in a tabular

format so that information can be viewed easily. The **Record Deletion Module** allows unwanted or incorrect records to be removed from the database. The **Database Management Module** handles the connection between the application and the SQLite database and performs operations such as inserting, retrieving, updating, and deleting records.

8. Database Design

The project uses an SQLite database named hospital.db. The required database table or tables are created automatically when the application is executed. Patient information is stored in a structured format so that records can be retrieved and managed efficiently. SQLite is suitable for this project because it is lightweight, requires no separate database server, and stores information locally in a database file.

The use of a database ensures that information entered by the user is not lost when the application is closed. When the program is opened again, previously stored records can be retrieved from the database. The database design also allows the application to perform basic CRUD operations, namely Create, Read, Update, and Delete.

9. Working of the System

When the Python program is launched, the Tkinter graphical user interface is displayed on the screen. At the same time, the application establishes a connection with the SQLite database. The user can then enter patient information through the available input fields. When a button such as Add, Search, Update, or Delete is selected, the corresponding operation is performed on the database.

After an operation is completed, the application updates the information displayed in the table. For example, when a new patient is registered, the record is inserted into the SQLite database and displayed in the record table. When a search operation is performed, the application retrieves matching information from the database and displays it to the user. Similarly, records can be updated or deleted according to the available functionality. When the application is closed, the database remains stored on the computer, allowing the records to be accessed again when the program is reopened.

10. Inputs and Outputs

The inputs provided by the user consist of patient-related information such as patient name, patient ID, phone number, diagnosis, doctor-related details, and other fields available in the application. The user enters this information through the graphical interface.

The main output of the system is the display of patient records in a tabular format. The application also provides the results of search operations and reflects changes made

through registration, updating, or deletion. These outputs allow users to view and manage stored information conveniently.

11. Advantages and Limitations

The Hospital Patient Management System provides several advantages. It has a simple graphical interface that makes basic record management easy to perform. Patient information is stored locally in an SQLite database, which allows quick retrieval and avoids the need for a separate database server. The system is easy to install and run on a standard computer with Python. It also reduces repetitive manual record handling and provides a useful demonstration of database-based application development.

However, the system has some limitations. It is primarily intended for small-scale use and does not include an advanced authentication or authorization system. Since the database is stored locally, the application does not provide cloud-based or multi-user synchronization. Data validation is also limited compared with professional hospital management software. Furthermore, the current version does not include advanced modules such as online appointment management, billing, prescription management, or detailed audit logging.

12. Future Enhancements

The system can be improved by adding several advanced features. An appointment scheduling module could be introduced to manage patient appointments and doctor availability. A doctor-management module could maintain detailed information about doctors and their departments. A billing module could be added to calculate and store patient charges. Prescription records could also be maintained digitally.

Security can be improved by introducing user authentication, role-based access, and audit logs. The application could also be connected to a centralized or cloud-based database to support multiple users and computers. Additional reporting and data-export facilities could be introduced to generate useful reports from the stored patient information.

13. Testing

Testing is performed to verify that the main functions of the application work correctly. The first test is to launch the application and confirm that the main graphical interface opens successfully. The second test involves entering a valid patient record and checking whether the record is saved and displayed correctly in the table. Another test verifies that the main database operation produces the expected changes in both the database and the displayed information. Finally, the application is closed and reopened to confirm that previously saved records remain available.

These tests help verify the basic functionality and reliability of the application. Successful completion of the tests indicates that the main objectives of the project have been achieved.

14. Conclusion

The Hospital Patient Management System successfully demonstrates how Python, Tkinter, and SQLite can be used together to develop a functional desktop-based management application. The system provides basic facilities for registering, searching, displaying, and managing patient information through a simple graphical interface. By storing information in an SQLite database, the application provides persistent local storage and makes record retrieval more convenient than traditional manual methods.

Although the current system is designed for small-scale and academic purposes, it provides a useful foundation for developing a more comprehensive hospital management solution. Additional features such as appointment scheduling, doctor management, billing, prescriptions, authentication, audit logs, and centralized database support can be incorporated in future versions. Overall, the project demonstrates the practical application of Python programming, GUI development, and database management concepts in solving a real-world record-management problem.

15. References / Learning Sources

The project was developed with the help of the Python Standard Library documentation, Tkinter documentation, SQLite documentation, and Python SQLite3 module documentation. These resources provide information about Python programming, graphical user interface development, database connectivity, SQL operations, and SQLite database management.

16. Project Files

The main project files include `patient_management.py`, which contains the Python source code; `README.txt`, which provides setup and usage instructions; `PROJECT_REPORT.txt`, which contains the project documentation; and `hospital.db`, which is the SQLite database file generated by the application.